Config Files

ONE LOVE. ONE FUTURE.

1

Reproducibility in Experiments

- Docker ensures reproducible compute environments.
- Other factors are essential for reproducibility.
- **Key Factor:** Specifying hyperparameters precisely.
- Example from this paper — results of 255 papers examined.

ĐẠI HỌC BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

2

Importance of Configuring Experiments

- Deep learning involves many hyperparameters.
- A *hyperparameter* influences the learning process but isn’t learned (e.g., weights are not hyperparameters).
- If hyperparameters are unstructured, it can be hard to track them in experiments.
- Configuration management ensures:
  - Reliability
  - Scalability

Basic Hyperparameter Structuring

Example (hardcoding hyperparameters):

```
class my_hp:
    batch_size: 64
    lr: 128
    other_hp: 12345

dl = DataLoader(Dataset,
batch_size=my_hp.batch_size)
```

- Issue: Configuration changes require modifying the script.

Using Argument Parsers

Example (configuring hyperparameters via command line):

```
python train.py --batch_size 256 --learning_rate 1e-
4 --other_hp 12345
```

- Better configurability, but experiment tracking can still be lost.

Introducing Hydra for Configuration Management

- Hydra + OmegaConf solve the problem.
- Configuration is moved to a `.yaml` file, simplifying version control.

Example `config.yaml`:

```
hyperparameters:
  batch_size: 64
  learning_rate: 1e-4
```

Python code for loading:

```
from omegaconf import OmegaConf
config = OmegaConf.load('config.yaml')
dl = DataLoader(dataset,
batch_size=config.hyperparameters.batch_size)
optimizer = torch.optim.Adam(model.parameters(),
lr=config['hyperparameters']['learning_rate'])
```

Using Hydra

```
import hydra

@hydra.main(config_name="basic.yaml")
def main(cfg):
    print(cfg.hyperparameters.batch_size,
cfg.hyperparameters.learning_rate)

if __name__ == "__main__":
    main()
```

- Hydra decouples configuration from code for easier version control.

ĐẠI HỌC BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

7

Exercises

Main Idea

- Use Hydra to log experiment configurations precisely.
- Provided script: hyperparameters hardcoded—your task is to separate them into a config file.

ĐẠI HỌC BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

8

Steps

1. Install Hydra: `pip install hydra-core --upgrade`
2. Review `vae_mnist.py` and `model.py`.
3. Identify key hyperparameters. Consider randomness in initialization.
4. Write `config.yaml` for the hyperparameters.
5. Load the config file in `vae_mnist.py` (use Hydra).
6. Run the script.
7. Inspect Hydra's output folder for logged hyperparameters.
8. Dynamically change/add parameters from the command line:
   - Change:
     ```
     python vae_mnist.py hyperparameters.seed=1234
     ```
   - Add:
     ```
     python vae_mnist.py +experiment.stuff_that_i_want_to_add=42
     ```
9. Ensure terminal outputs are logged:
   - Replace `print` with `log.info`:
     ```
     import logging
     log = logging.getLogger(__name__)
     ```
10. Check reproducibility with multiple runs: bash `python reproducibility_tester.py path/to/run/1 path/to/run/2`

More Exercise

- Make your MNIST code fully reproducible.
- Use multiple config files (e.g., `model_conf.yaml` and `training_conf.yaml`).
- Possible structure:

```
|--conf
|    |--config.yaml
|    |--experiments
|         |--exp1.yaml
|         |--exp2.yaml
|--my_app.py
```

Example: Complex Configuration Setup

- Separate model hyperparameters and optimizer hyperparameters.
- Supports different architectures and optimizers.

```
├── conf
│   ├── config.yaml
│   ├── dataset
│   │   ├── cifar10.yaml
│   │   └── imagenet.yaml
│   └── optimizer
│       ├── adam.yaml
│       └── nesterov.yaml
└── my_app.py
```


Benefits of YAML Configuration

- Refactoring hyperparameters into .yaml files disentangles model from configuration.
- Easier to do version control of configurations in separate files.